

Structural and Functional Modeling in Event-Driven Process Methodology (EPM) Notation As a Tool for Generation of Program Code Simulating Process Activities

L. A. Mylnikov^{a, b, *} and A. D. Saltykova^{a, **}

^a National Research University Higher School of Economics, Perm, Russia

^b Perm National Research Polytechnic University, Perm, Russia

*e-mail: LAMylnikov@hse.ru

**e-mail: ADSaltykova@hse.ru

Received September 6, 2024

Abstract—This paper addresses the problem of the objective assessment of information processing effectiveness based on digital twins. A method was designed to establish correspondence between structural and functional models in event-driven process methodology notation and a symbolic description that can be transformed to a programming language based on a group of rules is described. Based on a given structural and functional model, examples of symbolic description and the R programming language are given. The results of the information processing research using this statistical analysis technique are presented.

Keywords: EPM, event-driven process methodology, simulation modeling, statistical study, process effectiveness evaluation, functional programming

DOI: 10.3103/S0005105524700389

INTRODUCTION

The efficiency of using information technologies varies across different areas of activity due to the use of specialized software as well as some functional capabilities (level of automation), methodology and labor intensity of use, requirements for user qualifications, established traditions of using IT solutions, and the form of data presentation in management/production/technological processes inherent in each subject area.

To describe process activities, structural and functional models are used in notations such as BPMN [1], eEPC [2], and IDEF [3], which are diagrams that have selected elements (actions, resources) and connections between them. The more modern notations ArchiMate (TOGAF) [4], OPM [5], and SysML [6] allow the relationship between process activities and the means of their implementation and support to be shown in a single diagram and to designate target performance indicators.

The study of the efficiency of process activities using the allocated key point indicators (KPI) and process performance indicators (PPI) is usually carried out by collecting statistics on already implemented processes, which opens the way to their reconstruction [7].

A study conducted by McKinsey [8] shows that companies that effectively apply methods for optimizing ongoing processes achieve up to a 30% reduction

in operating costs, improve productivity by 35%, and increase customer satisfaction by an average of 30%.

It is possible to analyze the efficiency of processes before their implementation (Fig. 1) by only simulating their operation on structural and functional models. Petri nets [9] have the necessary functionality for this, but they have a limitation associated with the implementation of requests/applications/queries one at a time. Simulation modeling using specialized tools and languages (Simulink, AnyLogic, SciLab, Simula, Algol) does not have an established standard for graphic notations and a universal programming language for describing models, which complicates their reading and creation.

Approaches to the study of structural and functional models such as AS-IS->TO-BE or the BPM [10] and CMMN [11] methodologies are of an expert nature and do not allow objective data on the effectiveness of certain processes to be obtained.

Unlike the above-mentioned descriptions and approaches, the event-driven process methodology (EPM) notation explicitly identifies blocks that define the control action (S), blocks of the environment (NV), the result (R), and the action (V), allowing one to set initial conditions, condition blocks, functional descriptions to represent the model in a symbolic form, which meets the requirements for functional programming languages.

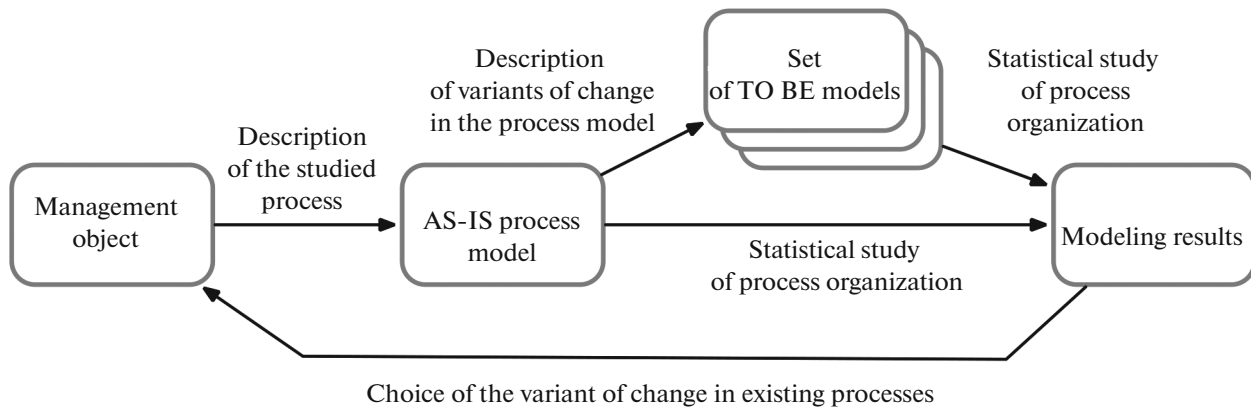


Fig. 1. Scheme for selecting a method for organizing process activity based on objective data.

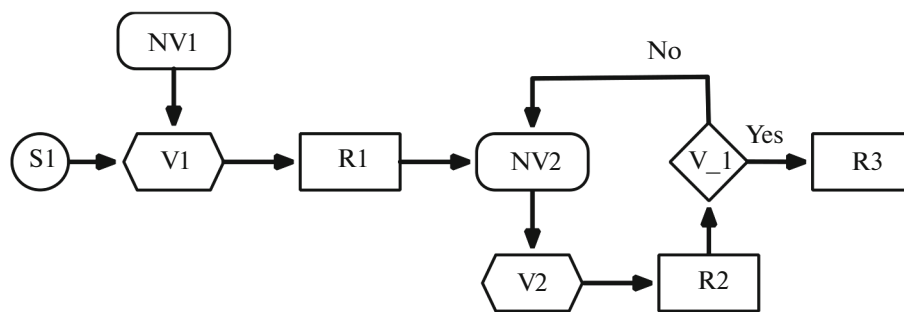


Fig. 2. The process of making a toast: S1 is a flow of requests for making a toast, V1 is preparatory operations (determine the degree of roasting, take out bread), V2 is making a toast, V_1 is checking the readiness of the toast, R2 is toast, R3 is the ready toast.

The availability of a method for formalizing a model in a pseudo-language allows one to switch to various programming languages—both universal and specialized—to model, conduct any statistical studies, and obtain objective assessments of efficiency based on simulation modeling relying upon the central limit theorem [12].

Obtaining objective assessments, in turn, opens the way to using methods for selecting solutions based on both the involvement of experts and the use of criteria-based selection.

METHODS

To study structural-functional models by simulating their operation, it is necessary to establish correspondences between schematic and sign-symbolic representations. For the EPM notation [13], this correspondence can be established as shown in Table 1.

Using a sign-symbolic description allows one to enter initial conditions (items 1 and 4 in Table 1), basic functions (items 2 and 3 in Table 1), conditions and cycles (item 5 in Table 1). To exchange data and through connections, it is necessary to use operators (see item 6 in Table 1).

The rules for constructing models are described in [13]: (1) using action blocks (V) in conjunction with

the S, NV (optional) and R blocks, (2) only the R block can be included in the condition block and the outputs of this block can be connected only to the R or NV blocks.

Taking the notation rules into account, the introduced correspondences between the structural-functional and symbolic representations allow one to formalize the writing of programs for studying the efficiency of processes. For example, if we consider the preparation of toasts in [14] and in Fig. 2, then by comparing the graphic elements of the symbolic representation, we obtain the symbolic description given in Listing 1.

Listing 1. Symbolic description corresponding to the structural-functional model shown in Fig. 2:

```

S1= Prob(<number of values >,
< probability >);
R1=V(<computational complexity >,
S1, NV1);
NV2=R1;
R2= V(<computational complexity >,
1, NV2);
do V_1(R2) then
R2=V(<computational complexity >,
1, NV2+R2);
else R3=R2.
  
```

Table 1. EPM notation artifacts, their functionality, purpose, sign-symbolic representation

No.	Schematic designation	Functionality	Purpose	Sign-symbolic description
1	 Initialization block (S)	Generates a simple natural number using a Poisson flow with a given probability or a periodically repeating value	Activation of the action block, generation of the number of requests/applications/calls	$S = \text{Prob}(\langle \text{number of values} \rangle, \langle \text{probability} \rangle)$ $S = \text{Periodic}(\langle \text{number of values} \rangle, \langle \text{period} \rangle)$
2	 Action block (V)	Implements an operation that will result in data or a delay associated with computational complexity, in which case the result will be requests that will appear at the output after some time	Request processing	$R = V(\langle \text{complexity} \rangle, S, NV)$ $R = F(S, NV)$ —for the case of an independently implemented function
3	 Result block (R)	Storage and transmission of the result to the next block at the next clock cycle by existing connection	Storage of the result of the action block (V)	
4	 Environment block (NV)	Stores data necessary for the action block (V)	Storage of data necessary for the model	$NV = R$ $NV\$ \langle \text{variable name} \rangle = \langle \text{value} \rangle$
5	 Condition block	Used to wait for actions to be performed (synchronization and control of the sequence of processes)	Implementation of cycles and conditional transitions	do $\langle \text{condition} \rangle$ then $\langle \text{action} \rangle$ else $\langle \text{action} \rangle$
6		Assignment	Data transfer	= means assigning a value to the block that the arrow enters + means combining data when multiple arrows enter; $\cap$ means selecting data when combining the results of several passes in one block when organizing cycles

The resulting representation uses vectors of values, which, on the one hand, is an advantage, since it allows data processing to take place in one pass, and on the other hand, requires revising the data specified by the S blocks to obtain statistically significant results with repeated calls and cycles, if randomly generated numbers were used to determine them (*Prob* function).

Let us explain this using an example that requires a specified number of repetitions or a change in the logic of operation in relation to the pass number. To account for the number of passes, it is necessary to use a global repetition counter (i): if some operational block must be executed under certain conditions or repeatedly (for example, twice in a row), and the data entering the action block is considered as requests requiring processing, then such a situation can be described by the structural-functional model shown in Fig. 3 [15].

In sign-symbolic form, such a structural-functional model (see Fig. 3) can be represented by Listing 2.

Listing 2. Sign-symbolic description corresponding to the structural-functional model shown in Fig. 3:

```

S1= Prob(<number of values >,
< probability >);
i=1;
R1(i)=V(<computational complexity >,
S1, NULL);
do V2(i) then
i=i+1;
S1= Prob(<number of values >,
< probability >);
R1(i)=V(<computational complexity >,
S1+R1, NULL);
else R2=R1(i)∩R1(i-1).

```

A feature associated with the operation of the model shown in Fig. 3 is that with each new use, the values of S1 must be updated.

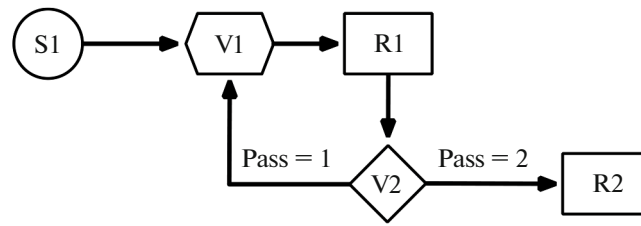


Fig. 3. An example of a structural and functional model with a repeating block in the process of processing an application: S1 is a block for generating a flow of applications (initial vector of requests), V1 is a block for processing applications, R1 is the results of processing applications in block V1, V2 is a conditional block used to check how many times applications have passed through block V1, R2 is the results of processing applications.

If repetitions affect the environment, we get the situation shown in Fig. 4.

In sign-symbolic form, the operation of the structural-functional model shown in Fig. 4 can be represented by Listing 3:

Listing 3. Sign-symbolic description corresponding to the structural-functional model shown in Fig. 4:

```

S1=Prob(<number of values>, <probability>);
i=1;
R1(i)=V(<computational complexity>, NV1);
do V2(i) then
  i=i+1;
  S1= Prob(<number of values>, <probability>);
  R1(i)=V(<computational complexity>, S1, R1);
else R2=R1(i).
  
```

The resulting sign-symbolic representations satisfy the requirements for the functional programming languages [16], which means that a correspondence can be established between the resulting sign-symbolic representation and other programming languages (including imperative ones) (the necessary functions

and operations can be implemented in another language, and parts of the code in the sign-symbolic notation can be matched with code in another programming language) and, when adding commands for collecting statistics, displaying them on the screen or in a file for plotting graphs, an analysis of the operation of processes described by structural-functional models can be performed.

STUDY OF SIGN-SYMBOLIC MODELS

To obtain a program in an existing programming language, it is necessary to implement the functions *Prob*, *Periodic*, *V*, *F* (additional functions implementing the operation of the algorithm for the element *V*). An example of the implementation of the *Prob* function in the R language is given in Listing 4. All values of the flow of applications/requests/calls are processed in one pass, since variables such as *S*, *R*, *NV* are vectors.

Listing 4. R implementation of a function that produces a set of random values, where the parameter *P* is the probability, *N* is the number of elements to fill:

```

S_prob<-function(N, P, ID){
  RES<-vector(mode = "integer",
length = N)
  RES[1:N]<-rpois(N,P)
  S1<-data.frame(S=RES, ID=vector
(mode = "numeric", length = N))
  ID_S<-ID
  for (i in 1:N){
    if (RES[i]>0){
      Vect<-as.vector(ID_S+1:RES[i],
mode = "numeric")
      ID_S<-ID_S+RES[i]
      S1$ID[i]<-list(Value=Vect)
    }
  }
  S_prob<-S1
}
  
```

In addition, the storage of data in variables, plotting graphs, displaying them on the screen, and saving

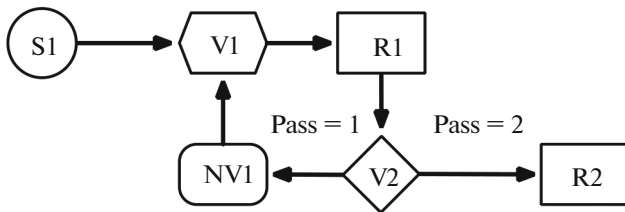


Fig. 4. An example of a structural-functional model with a repeating block in the process of processing a request: S1 is a block for generating a request flow (initial request vector), V1 is a block for processing requests, R1 is the results of request processing in block V1, V2 is a conditional block used to check how many times requests have passed through block V1, R2 is the results of request processing.

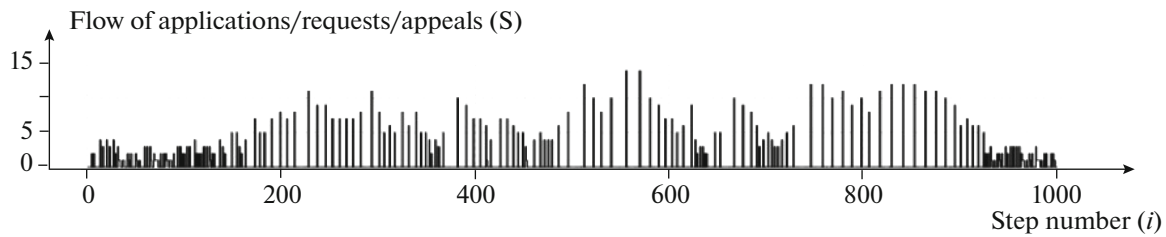


Fig. 5. Request flow when executing the program given in Listing 7, simulating the operation of the structural-functional model shown in Fig. 2.

an external file may require additional typical parts of code that can also be implemented using the function mechanism in the programming language used for implementation (see, for example, listings 5 and 6).

Listing 5. The part of the code that performs the data structure transformation after receiving the result *R* and collecting statistics in the variable *X* in the R language:

```
NV <- subset(R, select=c(R, ID_Out))
colnames(NV) <- c('S', 'ID')
X2 <- XES(R)
X <- rbind(X, X2)
```

Listing 6. The part of the code that allows, based on collected statistics (variable *X*), the determination of the time it takes for an application/request/access to pass through the process (variable *s*) and to present the obtained data as a violin diagram in the R language:

```
l<-unique(X$ID)
s<-NA
for (i in 1:length(l)){
  s[i]<-sum(X$W[X$ID==l[i]])
}
vioplot(s, col = "lightgray",
panel.first=grid())
```

The “if ... then” “else ...” or “while ... do ...” constructions, which are present in most programming languages, can be used to implement the conditional block.

Using the replacement rule based on correspondences, one can write program code that will be close in appearance to the sign-symbolic representation (Listing 7).

Listing 7. Program for simulating the operation of the model shown in Fig. 2:

```
N<-1000 # number of calculated
model steps
# Generate a stream of requests for
toast preparation (ID)
S<-S_prob(N, 0.9, 1000)
# Model the passage of events
through block V1
R1<-V(1, S, "V1")
```

```
# Collect statistics on the block
operation and the passage of projects
through it
```

```
X<-XES(R1)
#NV2=R1 (including removing unnec-
essary data)
```

```
NV2<-subset(R1, select=c(R, ID_Out))
colnames(NV2) <- c('S', 'ID')
```

```
R2<-V(1, NV2, "V2")
```

```
NV3<-subset(R2, select=c(R, ID_Out))
colnames(NV3) <- c('S', 'ID')
```

```
X2<-XES(R2)
```

```
X<-rbind(X,X2)
```

```
# Generate a random number (0 or 1)
if (sample(0:1, 1)) { # If 1 is
rolled, execute this block of code
```

```
R2 <- V(1, NV2, "V2")
```

```
NV3 <- subset(R2, select=c(R,
ID_Out))
```

```
colnames(NV3) <- c('S', 'ID')
```

```
X2 <- XES(R2)
```

```
X <- rbind(X, X2)
```

```
}
```

```
R3<-R2
```

```
# Total time to complete the toast
```

```
l<-unique(X$ID)
```

```
s<-NA
```

```
for (i in 1:length(l)){
```

```
  s[i]<-sum(X$W[X$ID==l[i]])
```

```
}
```

Thus, we obtain access to the data collected during the simulation statistics, which can be used to analyze the effectiveness of the structural-functional model (Figs. 5 and 6).

The resulting sign-symbolic representation allows us to study complex processes that cannot be implemented using other notations, for example, processes with different types of requests and processes using a common base (functionality, equipment) for their implementation (for example, the structural-functional model shown in Fig. 7).

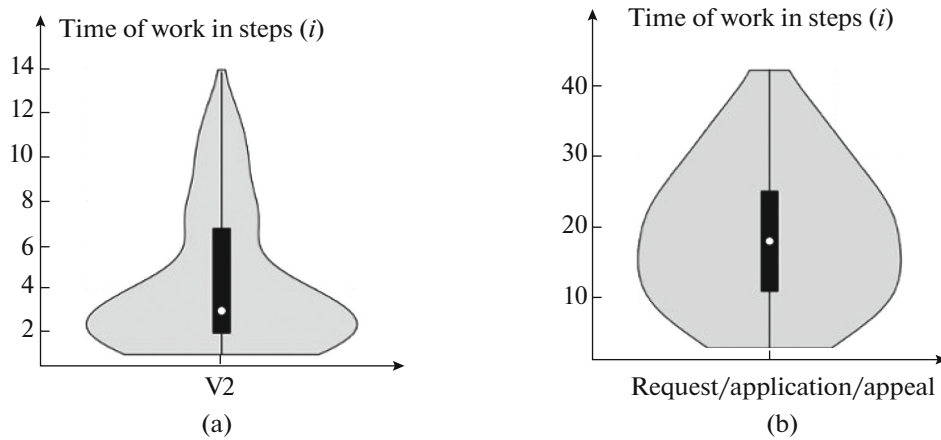


Fig. 6. Statistical data on the operation of the structural-functional model shown in Fig. 2: (a) block V2 loading time in work steps (i), (b) request processing time.

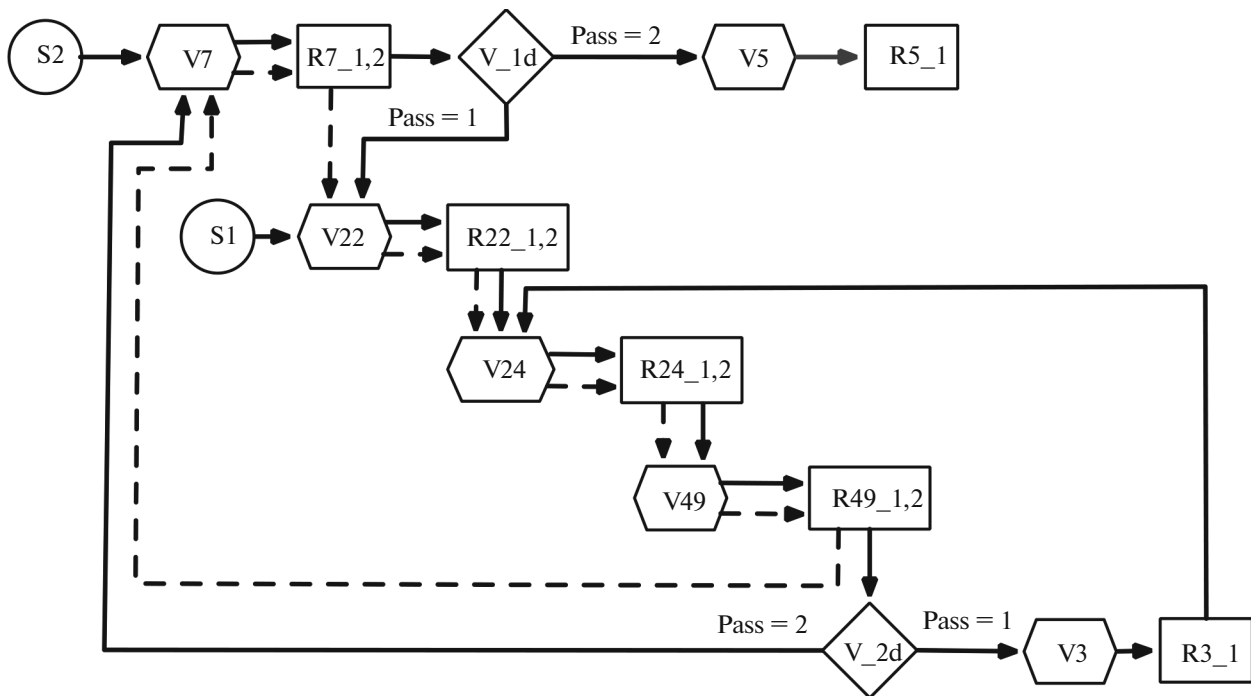


Fig. 7. Structural and functional model of the rotor (black color, one type of applications) and core (dashed line, another type of applications) painting process: S2—rotor before processing, S1—core before processing, V7—inspection, V22—degreasing, V24—painting, V49—drying, V3—cleaning, V5—coating defect correction, R22_1,2—core/rotor after degreasing, R24_1,2—core/rotor after painting, R49_1,2—core/rotor that has undergone the drying procedure, R3_1—rotor after cleaning, R7_1,2—core/rotor after inspection, R5_1—rotor after coating defect correction, V_1d—condition block (pass 1 = after the inspection operation the rotor is sent to the degreasing procedure, pass 2 = after the inspection operation the rotor is sent to the coating defect correction operation), V_2d—condition block (pass 1 = after the drying operation the rotor is sent to the cleaning procedure, pass 2 = after the drying operation the rotor is sent to the control procedure).

The approach under consideration allows us to build a sign-symbolic correspondence for such cases and obtain modeling results taking into account the mutual influence of different types of requests and with the joint use of action blocks (V) (for example,

loading schedules for each block of operations, Fig. 8, and the time of servicing requests by type, Fig. 9).

As can be seen from the above reasoning, the method of transition from structural-functional models to a sign-symbolic description and then to a pro-

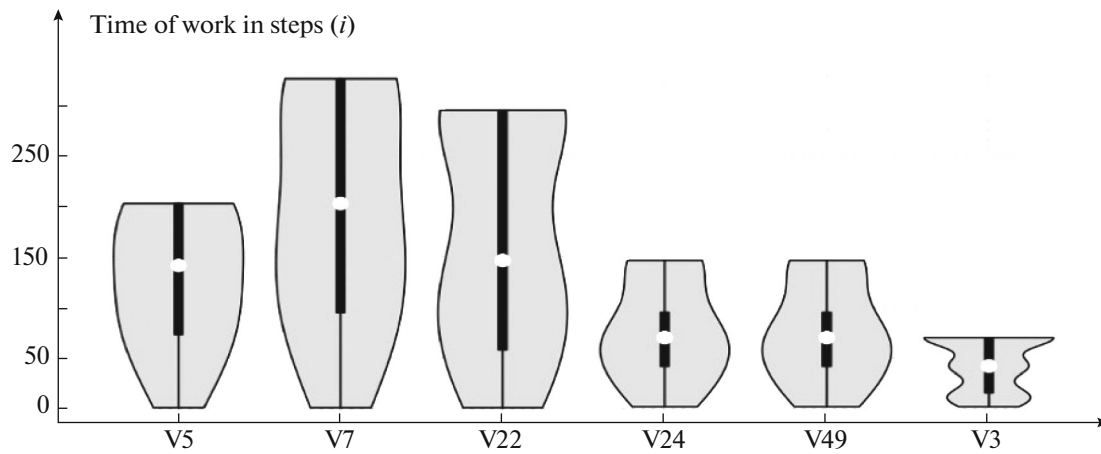


Fig. 8. Violin diagrams of the loading time of operation blocks with probabilities of occurrence of requests of the types $P(S1) = 0.9$ and $P(S2) = 0.9$.

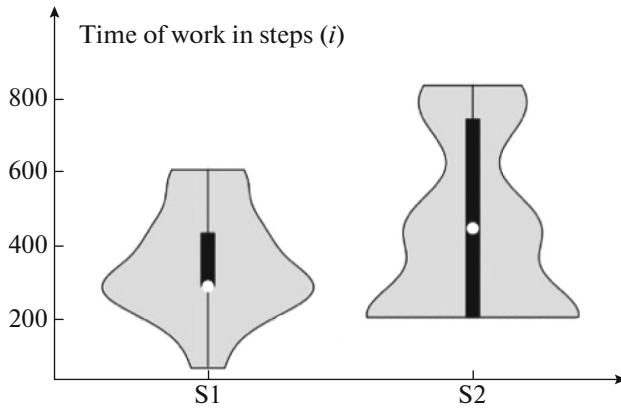


Fig. 9. Violin diagrams of the processing time of requests of the types S1 and S2 with probabilities of occurrence of requests of the types $P(S1) = 0.9$ and $P(S2) = 0.9$.

gram code does not impose restrictions on the type and structure of the processes under consideration, which allows us to study information processes, production processes, processes associated with activity, etc. using simulation modeling methods.

In this case, having built a structural and functional model, we can automatically obtain a symbolic description, a program code for modeling, and modeling data (data and graphs).

A limitation of the approach adopted used is the internal time, which is represented in steps/cycles of calculation (i), which allows work to be done only with discrete systems and indicates the need to establish a correspondence between the calculation steps and real time (this can be done based on information about the performance of the equipment and algorithms used).

For structural and functional models in the EPM notation, a correspondence with a symbolic descrip-

tion can be established and translated into an existing programming language. Sign-symbolic representation allows the efficiency of process activity to be studied using statistical analysis methods, comparing the implementation options for these processes based on the data obtained as a result of simulation modeling.

FUNDING

This study was supported by the Government of the Perm krai, project no. C-26/692.

CONFLICT OF INTEREST

The authors of this work declare that they have no conflicts of interest.

REFERENCES

1. Business Process Model and Notation (BPMN). Object Management Group, 2013. <https://www.omg.org/bpmn/>.
2. Scheer, A.-W., *ARIS—Modellierungsmethoden, Meta-modelle, Anwendungen*, Berlin: Springer, 2001.
3. Marca, D. and McGowan, C.L., *SADT: Structured Analysis and Design Technique*, New York: McGraw Hill, 1988.
4. ArchiMate® 3.1 Specification, The Open Group, 2019. <https://pubs.opengroup.org/architecture/archimate31-doc/copyright.html>.
5. Dori, D., *Object-Process Methodology: A Holistic Systems Paradigm*, Berlin: Springer, 2002. <https://doi.org/10.1007/978-3-642-56209-9>
6. Delligatti, L., *SysML Distilled: A Brief Guide to the Systems Modeling Language*, Upper Saddle River, NJ: Addison-Wesley, 2013.
7. van der Aalst, W., *Process Mining: Data Science in Action*, Berlin: Springer, 2016. <https://doi.org/10.1007/978-3-662-49851-4>
8. Morgan, J., *The Future Leader: 9 Skills and Mindsets to Succeed in the Next Decade*, Wiley, 2020.

9. Peterson, J.L., *Petri Net Theory and the Modeling of Systems*, Prentice-Hall, 1981.
10. Estruch, A. and Heredia Álvaro, J.A., Event-driven manufacturing process management approach, *Business Process Management. BPM 2012*, Barros, A., Gal, A., and Kindler, E., Eds., Lecture Notes in Computer Science, Berlin: Springer, 2012, pp. 120–133.
https://doi.org/10.1007/978-3-642-32885-5_9
11. Routis, I., Bardaki, C., Dede, G., Nikolaidou, M., Kamalakis, T., and Anagnostopoulos, D., CMMN evaluation: The modelers' perceptions of the main notation elements, *Software Syst. Model.*, 2021, vol. 20, no. 6, pp. 2089–2109.
<https://doi.org/10.1007/s10270-021-00880-3>
12. Doyle, P.G., *Grinstead and Snell's Introduction to Probability*, Providence, RI: American Mathematical Society, 2006.
13. Mylnikov, L.A., Saltykova, A.D., and Avramovic, Z., Event-driven process methodology notation for information processing research, *Autom. Doc. Math. Linguist.*, 2024, vol. 58, pp. 243–254.
<https://doi.org/10.3103/S0005105524700183>
14. Wujec, T., *Wicked Problem Solving (Practitioner Workbook)*, Toronto: Wujec Group, 2021.
15. Mylnikov, L.A., Structural and functional modeling of processes with a dedicated control subject, *Autom. Doc. Math. Linguist.*, 2022, vol. 56, no. 1, pp. 42–54.
<https://doi.org/10.3103/s0005105522010083>
16. Field, A.J. and Harrison, P.G., *Functional Programming*, Wokingham, England: Addison-Wesley, 1988.

Translated by L. Solovyova

Publisher's Note. Allerton Press remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.
AI tools may have been used in the translation or editing of this article.